# Software Requirements Specification

for

# Big O'ff

Version 1.0

**Prepared by**

Group Name: Team 45


| Name | Student # |
|---|---|
| Tejaansh | SE23UCSE174 |
| Rehan | SE23UARI101 |
| Aditya | SE23UCSE231 |
| Divyank | SE23UCSE056 |
| Dalip Inder Singh | SE23UARI030 |
| Rishi | SE23UARI103 |
| Talha Affan | SE23UARI144 |
| Nikhila | SE23UCSE214 |


| Field | Details |
|---|---|
| Instructor: | TBD |
| Course: | SOFTWARE ENGINEERING [CS3201] |
| Lab Section: | TBD |
| Teaching Assistant: | Mr Arun Avinash Chauhan |
| Date: | May 4 , 2026 |

## Revisions

| Version | Primary Author(s) | Description of Version | Date Completed |
|---|---|---|---|
| 1.0 | Team 45 | Initial SRS Draft | May 4 , 2026 |

# 1 Introduction

Big O'ff is a real-time competitive programming platform designed to transform the traditionally solitary grind of DSA (Data Structures & Algorithms) practice into a high-stakes, gamified battle arena. This document serves as the Software Requirements Specification (SRS) for the Big O'ff platform, outlining the functional, non-functional, and interface requirements that guide its design and development. The sections below describe the product's scope, intended audience, functional features, system interfaces, and quality attributes.

## 1.1 Document Purpose

This Software Requirements Specification defines the complete set of requirements for the Big O'ff platform — a real-time DSA battleground developed by Team 45. The document covers the core platform features including 1v1 combat mechanics, algorithmic complexity detection, sabotage systems, a live spectator layer, and user profile dashboards.

This SRS serves as the primary reference for the development team, ensuring all members share a unified understanding of what the system must do. It also acts as a baseline document for verification and validation activities throughout the project lifecycle.

## 1.2 Product Scope

Big O'ff is a web-based competitive programming platform that pits two players against each other in real-time coding duels across a Three-Round Gauntlet (Easy, Medium, Hard difficulty tiers). The platform introduces a layer of strategic depth through sabotage mechanics — timed actions that disrupt the opponent's workflow — and a live spectator experience with interactive pixel-art emotes.

The primary goal of the product is to disrupt the traditional, solitary culture of competitive programming by introducing excitement, community engagement, and psychological strategy into the coding process. Secondary benefits include gamified skill tracking, ELO-based ranking, and a GitHub-style contribution heatmap for visualizing long-term growth.

## 1.3 Intended Audience and Document Overview

This SRS is intended for the following reader groups:

- Development Team (Developers, Backend Engineers, UI/UX Designers): Should read all sections, with particular focus on Sections 3 and 4.
- Project Manager: Should read Sections 1, 2, and 4 for scope and constraint clarity.
- Course Instructor / Client: Should read Sections 1, 2, and 3 to validate alignment with project goals.
- Teaching Assistant: Should read all sections to evaluate completeness and correctness.

The document is organized as follows: Section 1 provides an introduction and context. Section 2 gives a high-level product overview. Section 3 details specific functional and interface requirements. Section 4 lists non-functional quality requirements. Appendices provide a data dictionary and group log.

## 1.4 Definitions, Acronyms and Abbreviations

| Term / Acronym | Definition |
|---|---|
| DSA | Data Structures and Algorithms |
| SRS | Software Requirements Specification |
| HUD | Heads-Up Display — the in-game UI overlay showing HP bars and battle status |
| HP | Health Points — numerical representation of a player's standing in a battle |
| ELO | A rating system used to calculate relative skill levels of players |
| IDE | Integrated Development Environment — the in-browser code editor |
| PubSub | Publish-Subscribe messaging pattern used for real-time event distribution |
| CRT | Cathode Ray Tube — retro scanline visual overlay used for aesthetic effect |
| API | Application Programming Interface |
| 1v1 | One-versus-one: a two-player competitive format |
| Big-O | Mathematical notation for algorithmic time/space complexity |
| UML | Unified Modeling Language — used for system design diagrams |
| COMET | Component-based Object and Multiagent sysTeems — software design method |

## 1.5 Document Conventions

This document follows IEEE SRS formatting standards. The following conventions apply:

- Font: Arial, size 11pt for body text.
- Headings follow a numbered hierarchical structure (e.g., 1, 1.1, 1.1.1).
- Functional requirements are prefixed with 'F' (e.g., F1, F2).
- Performance requirements are prefixed with 'P' (e.g., P1, P2).
- Safety/security requirements are prefixed with 'S' (e.g., S1, S2).
- Italicized text denotes instructional comments inherited from the template (not part of the final specification).
- All tables use consistent borders and shading for readability.

## 1.6 References and Acknowledgments


- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- COMET Method: Gomaa, H. (2011). Software Modeling and Design. Cambridge University Press.
- UML Specification: Object Management Group. https://www.omg.org/spec/UML/

- Clerk Authentication: https://clerk.com
- Django Framework: https://www.djangoproject.com
- Celery Task Queue: https://docs.celeryq.dev
- Redis: https://redis.io
- Statement of Work — Big O'ff, Team 45, February 13, 2026.

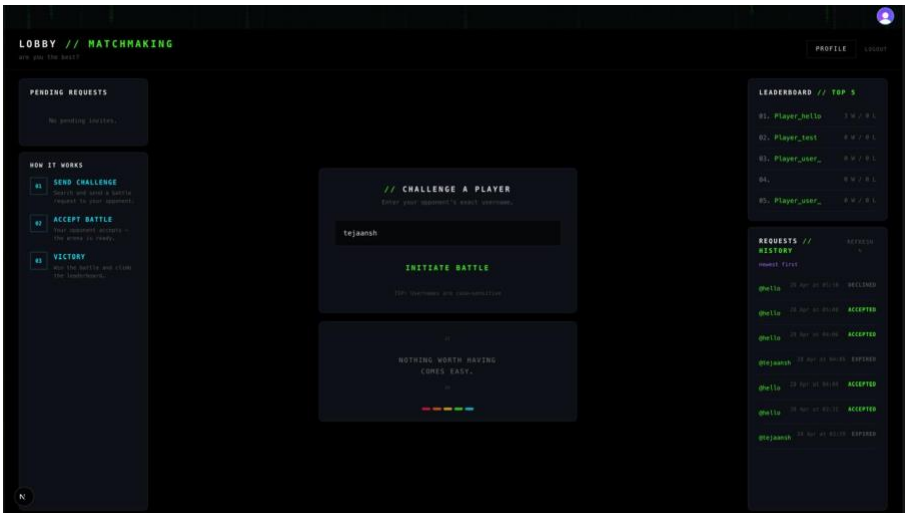
# 2 Overall Description

## 2.1 Product Overview

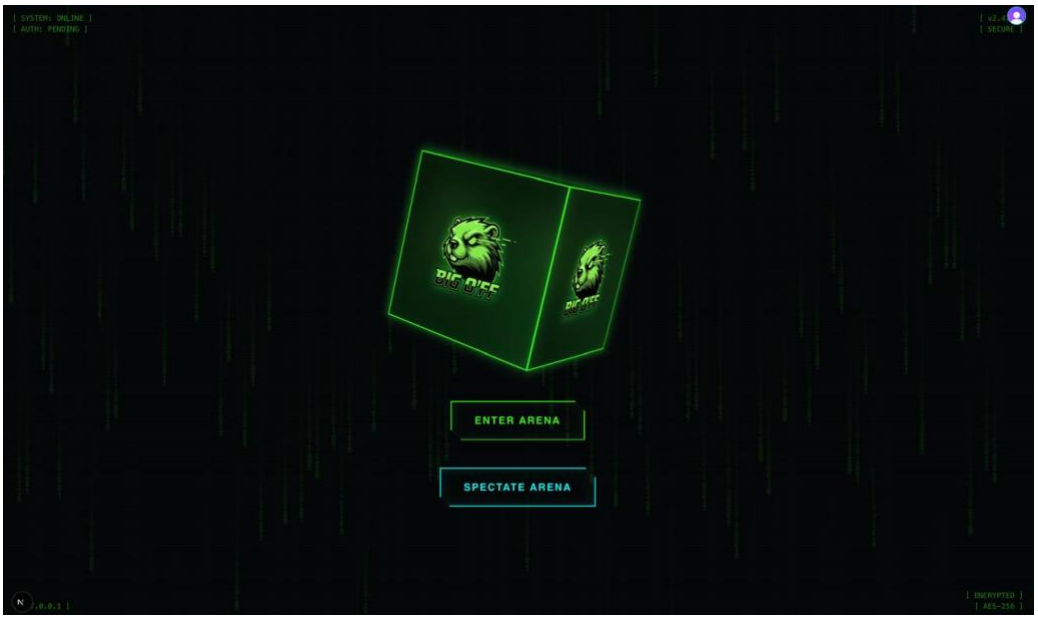
Big O'ff is a new, self-contained web platform with no predecessor product. It occupies a novel position in the competitive programming space by merging a real-time coding environment with game mechanics typically found in multiplayer combat games.

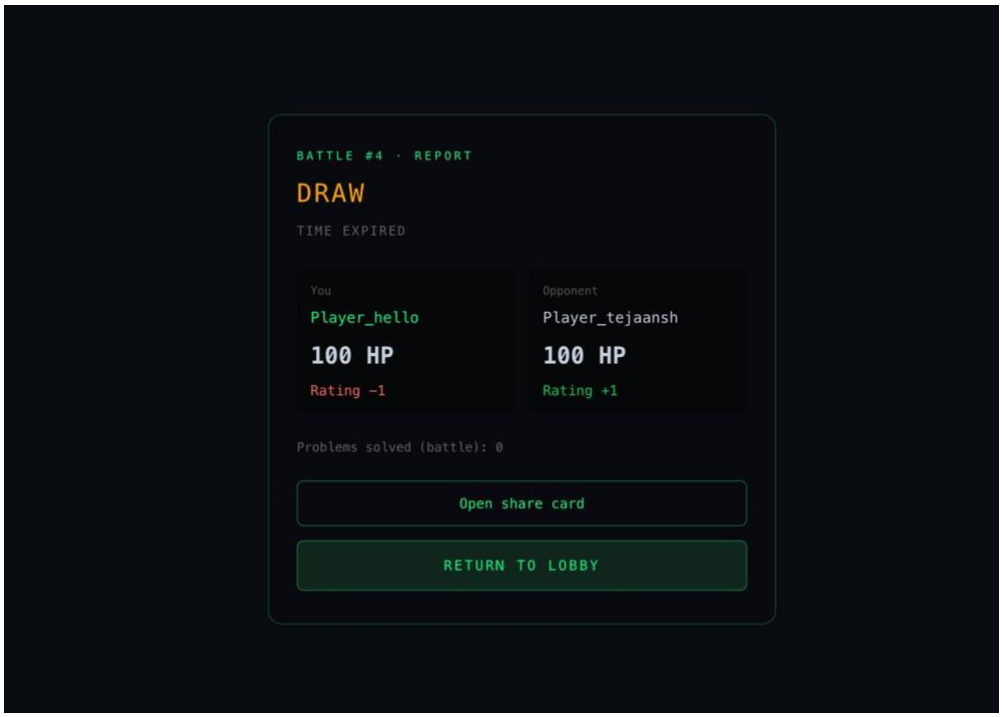
The system is composed of the following major subsystems:

- Battle Engine: Manages the 1v1 match lifecycle, round progression, and result determination.
- Algorithmic Efficiency Engine: Analyzes submitted code to determine Big-O complexity and scales HP damage accordingly.
- Sabotage System: Executes timed disruptive actions (e.g., screen-wipe, fog of war) against the opponent.
- Real-Time Synchronization Layer (Nervous System): A Redis-powered WebSocket PubSub system that keeps HP bars, sabotage events, and spectator feeds synchronized across all clients.
- Execution Sandbox: A Django + Celery dual-queue sandboxed subprocess environment for safe, low-latency code execution.
- Spectator Layer: A live sidebar featuring pixel-art emotes and streamed battle feeds viewable by non-participants.
- User Profile & Legacy System: Tracks ELO ratings, battle history, and displays a GitHub-style contribution heatmap.




SPECTATE LOBBY
Live battles //
Watch real players in action. Every match is live.
Filter
Newest ▾
← Home
BATTLE #6
LIVE
P
Player_tejaansh
vs
P
Player_hello
HP
100 / 100
100 / 100
Round 1
0 watching
WATCH >


[ SYSTEM: ONLINE ]
[ AUTH: PENDING ]
[ SECURE ]
ENTER ARENA
SPECTATE ARENA
[ ENCRYPTED ]
[ AES-256 ]


BATTLE #4 · REPORT
DRAW
TIME EXPIRED
You
Player_hello
100 HP
Rating −1
Opponent
Player_tejaansh
100 HP
Rating +1
Problems solved (battle): 0
Open share card
RETURN TO LOBBY

## 2.2 Product Functionality

The major functions of the Big O'ff platform are:

- User Authentication: Secure login via Google OAuth through Clerk.
- 1v1 Battle Matching: Players are matched and enter a Three-Round Gauntlet (Easy → Medium → Hard).
- Split-Screen Twin IDE: A dual-pane code editor displaying the player's code and a 'scrambled' view of the opponent's code.
- Real-Time Code Submission & Execution: Code is sandboxed, executed, and evaluated for correctness and algorithmic complexity.
- HP Bar System: Damage is dealt based on correctness and the relative Big-O complexity gap between solutions.
- Sabotage Mechanics: Players can deploy moves such as 'Garbage Collection' (screen-wipe) and 'Fog of War' (recurring screen disruption every 90 seconds).
- Live Spectator Layer: Non-players can watch live matches and send pixel-art emotes via a dedicated sidebar.
- User Profile Dashboard: Displays ELO rating, match history, and a GitHub-style contribution heatmap.
- Landing Page: Terminal-style animated hero section with Google Auth integration.

## 2.3 Design and Implementation Constraints

- Technology Stack: The backend must use Django (Python) and Celery for task queuing. Redis must be used as the message broker and WebSocket PubSub layer.
- Code Execution Safety: All user-submitted code must run in sandboxed subprocesses with strict CPU and memory resource limits to prevent system abuse.
- Real-Time Constraint: The system must support sub-millisecond event propagation for HP bar updates and sabotage triggers via the WebSocket layer.
- Authentication: All users must authenticate via Google OAuth. Clerk is the designated authentication provider.
- Design Methodology: The COMET (Component-based Object and Multiagent sysTeems) method must be used for software design. Ref: Gomaa (2011).
- Modeling Language: All system diagrams must be expressed in UML (Unified Modeling Language). Ref: OMG UML Specification.
- Atomic Transactions: HP deductions and sabotage effects must use atomic database transactions to ensure frame-perfect consistency.
- Frontend Aesthetic: The UI must adhere to the 'Terminal Noir' design language — jet blacks, neon greens, and CRT scanline overlays.

## 2.4 Assumptions and Dependencies

- All users will authenticate exclusively via Google accounts; no local username/password system is required.
- Players are assumed to have a stable internet connection sufficient to maintain a WebSocket connection throughout a match.
- The Clerk authentication service will remain available and compatible with the project's Google OAuth integration.
- Redis will be deployed as a managed service or self-hosted instance with sufficient throughput to handle concurrent WebSocket PubSub events.
- The server infrastructure will be capable of running Django, Celery workers, and Redis simultaneously.
- The Celery scheduling service will reliably trigger the 'Fog of War' sabotage task every 90 seconds.
- Third-party libraries and frameworks (Django, Celery, Redis, Clerk) will remain stable and supported throughout the development period.

# 3 Specific Requirements

## 3.1 External Interface Requirements

### 3.1.1 User Interfaces

The Big O'ff platform exposes the following user-facing interfaces:

- Landing Page: A hero section featuring terminal-style animations, a project description, and a 'Sign in with Google' button. The aesthetic follows Terminal Noir conventions (dark background, neon green text, monospace font).
- Battle HUD (Heads-Up Display): A split-screen IDE layout. The left pane displays the player's own code editor. The right pane shows a 'scrambled' (obfuscated) view of the opponent's code. Real-time HP bars are visible at the top of the screen for both players.
- Sabotage Controls: Contextual UI elements (buttons or keyboard shortcuts) to trigger available sabotage moves during a match.
- Spectator View: A read-only match stream with a live sidebar for submitting and viewing pixel-art emotes.
- User Profile / Stats Dashboard: Displays the user's ELO rating, match history, win/loss record, and a GitHub-style contribution heatmap showing activity across the calendar year.

### 3.1.2 Hardware Interfaces

The Big O'ff platform is a web-based application and does not directly interface with physical hardware components. The following hardware interactions are abstracted through the browser and server environment:

- Client Device (PC / Laptop / Desktop): Requires a modern web browser. Input via keyboard (code entry) and mouse/trackpad (UI navigation, sabotage triggers).
- Server Hardware: Hosts the Django application server, Celery worker processes, and Redis instance. Must support concurrent sandboxed subprocess execution.
- Network Interface: All communication between client and server occurs over HTTPS (REST API) and WSS (WebSocket Secure) protocols.

### 3.1.3 Software Interfaces

- Google OAuth 2.0 (via Clerk): Used for user authentication. The system sends authentication requests to Clerk's API, which interfaces with Google's identity services.
- Clerk Authentication SDK: Integrated into the frontend to manage session tokens and user identity.
- Redis: Acts as the WebSocket PubSub broker. Django Channels (or equivalent) publishes events (HP updates, sabotage triggers, emotes) to Redis; all subscribed clients receive updates in real time.
- Celery: Interfaces with the Django backend to schedule and execute asynchronous tasks, including the 'Fog of War' recurring sabotage (every 90 seconds) and code execution sandboxing.

- Browser WebSocket API: The frontend client maintains a persistent WebSocket connection to receive live game events.

## 3.2 Functional Requirements

### F1: User Authentication

The system shall allow users to register and log in exclusively via Google OAuth, managed through the Clerk authentication provider. Upon successful authentication, the system shall create or retrieve a user profile.

### F2: 1v1 Battle Matchmaking

The system shall match two authenticated users into a 1v1 battle session. Each battle shall consist of three rounds of increasing difficulty: Easy, Medium, and Hard.

### F3: Split-Screen Twin IDE

The system shall provide a split-screen in-browser code editor. The player's own editor shall be fully interactive. The opponent's code pane shall display a 'scrambled' (obfuscated) real-time view of the opponent's code.

### F4: Code Execution Sandbox

The system shall execute user-submitted code in an isolated sandboxed subprocess with strict CPU time limits and memory limits to prevent resource abuse or system exploitation.

### F5: Algorithmic Complexity Detection

The system shall analyze the execution metrics of submitted code to determine its Big-O time complexity. If a player's solution has a lower complexity class than the opponent's (e.g., O(n) vs O(n^2)), the system shall scale the HP damage dealt accordingly.

### F6: HP Bar System

The system shall maintain HP bars for both players. HP deductions shall be applied using atomic database transactions to ensure frame-perfect consistency. HP changes shall be propagated to all clients in real time via the WebSocket PubSub layer.

### F7: Sabotage Mechanics

The system shall implement the following sabotage moves:

- Garbage Collection: Blanks the opponent's code editor screen.
- Fog of War: A recurring Celery-scheduled task that applies a visual disruption effect to the opponent's screen every 90 seconds.

Sabotage actions shall be processed atomically to ensure HP deductions are consistent.

### F8: Live Spectator Layer

The system shall allow authenticated users not participating in a match to watch a live stream of the battle. Spectators shall be able to send pixel-art emotes via a dedicated sidebar, visible to all viewers and participants.

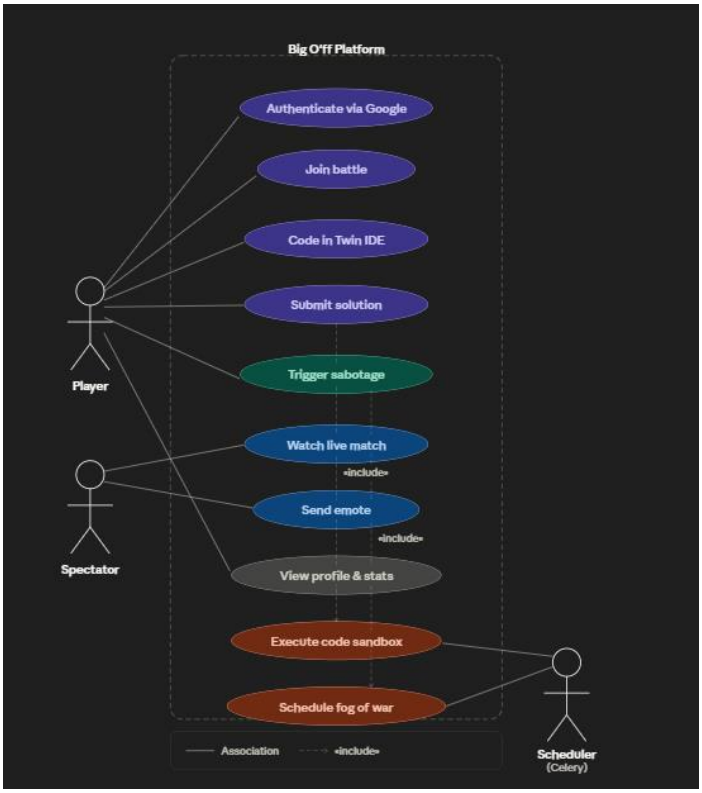
### F9: User Profile & Stats Dashboard

The system shall maintain a user profile tracking ELO rating, match history, and battle outcomes. The dashboard shall display a GitHub-style contribution heatmap showing the user's activity across the calendar year.

### F10: Landing Page

The system shall provide a landing page with terminal-style animations and an integrated 'Sign in with Google' button via Clerk.

## 3.3 Use Case Model




### Use Case U1: Authenticate User

| Field | Details |
|---|---|
| Author | Tejaansh |
| Purpose | Allow a user to securely log into the Big O'ff platform using their Google account via Clerk OAuth. |
| Requirements Traceability | F1 |
| Priority | High |
| Preconditions | The user has a valid Google account. The Clerk service is operational. |
| Post Conditions | The user is authenticated and a session token is issued. A user profile is created or retrieved. |
| Actors | User (Human), Google OAuth (External System), Clerk (External Service) |
| Extends | None |
| Basic Flow | 1. User navigates to the Landing Page. 2. User clicks 'Sign in with Google'. 3. System redirects to Google OAuth via Clerk. 4. User grants permissions. 5. Clerk returns a session token. 6. System creates or retrieves the user profile. 7. User is redirected to the home dashboard. |
| Alternative Flow | User denies Google permissions → System displays an error and returns to the Landing Page. |
| Exceptions | Clerk service unavailable → System displays a service error message. |
| Includes | None |
| Notes / Issues | All authentication is exclusively via Google. No local credentials are stored. |

### Use Case U2: Conduct 1v1 Battle

| Field | Details |
|---|---|
| Author | Rehan / Aditya |
| Purpose | Allow two players to compete in a real-time three-round coding battle with live HP tracking and sabotage mechanics. |
| Requirements Traceability | F2, F3, F4, F5, F6, F7 |
| Priority | High |
| Preconditions | Both players are authenticated. A match session has been created by the matchmaking system. |
| Post Conditions | The match concludes. HP results are recorded. ELO ratings are updated for both players. |
| Actors | Player 1 (Human), Player 2 (Human), Battle Engine (System), Execution Sandbox (System), Celery Scheduler (System) |

| Field | Details |
|---|---|
| Extends | None |
| Basic Flow | 1. Both players enter the Battle HUD. 2. Round 1 (Easy) problem is presented. 3. Players code their solutions in the Twin IDE. 4. A player submits code; the Sandbox executes it. 5. Complexity Engine determines Big-O; HP is deducted from the slower solution's player. 6. Players may deploy sabotage moves. 7. Rounds 2 (Medium) and 3 (Hard) follow. 8. The player with higher HP at match end wins. |
| Alternative Flow | A player disconnects mid-match → Opponent is awarded a win by default. |
| Exceptions | Sandbox timeout or crash → System marks the submission as failed; no HP change. |
| Includes | U3 (Execute Code), U4 (Trigger Sabotage) |
| Notes / Issues | Fog of War sabotage runs as a recurring Celery task every 90 seconds regardless of player action. |

### Use Case U3: Watch as Spectator

| Field | Details |
|---|---|
| Author | Dalip Inder Singh |
| Purpose | Allow non-participating authenticated users to watch a live battle and interact via emotes. |
| Requirements Traceability | F8 |
| Priority | Medium |
| Preconditions | A live match is in progress. The spectator is authenticated. |
| Post Conditions | Spectator has viewed the match feed and optionally submitted emotes visible to all viewers. |
| Actors | Spectator (Human), Battle Engine (System), WebSocket PubSub (System) |
| Extends | None |
| Basic Flow | 1. Spectator navigates to a live match. 2. System streams the live battle HUD to the spectator (read-only). 3. Spectator views real-time HP bars and code activity. 4. Spectator selects a pixel-art emote and submits it. 5. Emote is broadcast to all connected clients via PubSub. |
| Alternative Flow | None |
| Exceptions | WebSocket disconnection → Spectator view refreshes and reconnects automatically. |
| Includes | None |
| Notes / Issues | Spectators do not affect match outcomes. |

# 4 Other Non-Functional Requirements

## 4.1 Performance Requirements

- P1: The WebSocket PubSub layer shall propagate HP bar updates and sabotage events to all connected clients within 100 milliseconds under normal load conditions.
- P2: Code submitted to the Execution Sandbox shall begin execution within 500 milliseconds of submission receipt.
- P3: The 'Fog of War' Celery task shall be triggered within ±5 seconds of its scheduled 90-second interval.
- P4: The system shall support a minimum of 50 concurrent active battles (100 players) without degradation in WebSocket event delivery performance.
- P5: The landing page shall achieve a Time to First Contentful Paint (FCP) of under 2 seconds on a standard broadband connection.

## 4.2 Safety and Security Requirements

- S1: All user-submitted code shall execute within an isolated sandboxed subprocess. The sandbox shall enforce strict CPU time limits (maximum 5 seconds per submission) and memory limits (maximum 256 MB per submission) to prevent denial-of-service attacks.
- S2: All client-server communication shall occur over HTTPS and WSS (WebSocket Secure) protocols. Unencrypted HTTP connections shall be rejected.
- S3: User authentication tokens issued by Clerk shall be validated on every authenticated API request. Expired or invalid tokens shall result in a 401 Unauthorized response.
- S4: HP deductions and sabotage effects shall use atomic database transactions to prevent race conditions and ensure data integrity.
- S5: The spectator layer shall be read-only. Spectators shall have no ability to affect match state, submit code, or trigger sabotage actions.
- S6: User-submitted code shall not have access to the host filesystem, network, or any system resources outside the sandbox boundary.

## 4.3 Software Quality Attributes

### 4.3.1 Reliability

The system shall maintain a minimum uptime of 99% during active match periods. The WebSocket connection manager shall implement automatic reconnection logic so that transient network disruptions (under 30 seconds) do not terminate an active battle session. Celery workers shall be configured with retry logic for failed tasks to prevent sabotage events from being silently dropped.

### 4.3.2 Maintainability

The system shall be designed following the COMET method, decomposing functionality into well-defined components (Battle Engine, Execution Sandbox, Complexity Engine, PubSub Layer, etc.) with clearly specified interfaces. Each component shall be independently deployable

and testable. Code shall adhere to PEP 8 standards (Python) and documented with inline comments for all non-trivial logic.

### 4.3.3 Adaptability (Design for Change)

The Algorithmic Complexity Engine shall be implemented as a pluggable module, allowing new complexity detection algorithms to be swapped in without modifying the Battle Engine. The sabotage system shall use a configuration-driven approach so that new sabotage moves can be added by defining new Celery tasks and registering them in a central sabotage registry, without altering core match logic. The problem set (Easy/Medium/Hard questions) shall be stored in a database rather than hardcoded, allowing content updates without redeployment.

### 4.3.4 Usability

The Battle HUD shall present HP bars, round indicators, and sabotage controls in a manner that a first-time user can understand within 2 minutes of entering a match, without external documentation. The Terminal Noir aesthetic shall be consistently applied across all pages to create a cohesive and immersive user experience. The spectator emote system shall require no more than two clicks to submit an emote.

### 4.3.5 Testability

All sandboxed code execution paths shall be unit-testable via mock subprocess interfaces. The HP deduction and atomic transaction logic shall be covered by automated integration tests. The Celery task scheduler (Fog of War) shall be testable using Celery's eager execution mode in test environments, allowing task logic to be verified without a live Redis/Celery broker.

# 5 Other Requirements

5.1 Database Requirements: The system shall use a relational database (e.g., PostgreSQL) to store user profiles, ELO ratings, match history, and the problem set. All HP transactions shall be recorded with timestamps for audit purposes.

5.2 Internationalization: The initial release targets English-speaking users. The system architecture shall not hardcode string literals in the UI layer, allowing future localization without structural changes.

5.3 Legal: User-submitted code is the intellectual property of the submitting user. The platform shall not store, distribute, or reuse user code outside of the match execution context without explicit user consent.

## Appendix A – Data Dictionary

| Name | Type | Description | Possible Values / States | Related Requirements |
|---|---|---|---|---|
| user_id | String (UUID) | Unique identifier for an authenticated user | UUID v4 string | F1, F9 |
| elo_rating | Integer | Player's current ELO skill rating | Default: 1000; Range: 0–3000 | F9 |
| hp_player1 | Integer | Current HP of Player 1 in an active match | 0–100 | F6 |
| hp_player2 | Integer | Current HP of Player 2 in an active match | 0–100 | F6 |
| match_status | Enum | Current state of a battle session | WAITING, IN_PROGRESS, COMPLETED, ABORTED | F2, F6 |
| round_number | Integer | Current round within a battle | 1 (Easy), 2 (Medium), 3 (Hard) | F2 |
| complexity_score | Enum | Big-O complexity class of a submitted solution | O(1), O(log n), O(n), O(n log n), O(n^2), O(2^n), UNKNOWN | F5 |
| sabotage_type | Enum | Type of sabotage move deployed | GARBAGE_COLLECTION, FOG_OF_WAR | F7 |
| submission_status | Enum | Result of a code submission evaluation | PENDING, PASSED, FAILED, TIMEOUT, ERROR | F4 |
| spectator_emote | String | Identifier for a pixel-art emote sent by a spectator | Emote ID from emote registry | F8 |
| fog_interval_sec | Integer (Constant) | Interval in seconds between Fog of War triggers | 90 (fixed) | F7, P3 |

| Name | Type | Description | Possible Values / States | Related Requirements |
|---|---|---|---|---|
| sandbox_cpu_limit | Integer (Constant) | Maximum CPU execution time per submission (seconds) | 5 (fixed) | S1 |
| sandbox_mem_limit_mb | Integer (Constant) | Maximum memory per submission (MB) | 256 (fixed) | S1 |

## Appendix B – Group Log

| Date | Activity | Attendees | Notes |
|---|---|---|---|
| February 2, 2026 | Project Kickoff – Initial idea pitch and architecture design | All members | Agreed on Terminal Noir aesthetic, core feature set, and tech stack (Django, Celery, Redis, Clerk) |
| February 13, 2026 | Statement of Work completed and submitted | All members | Roles and responsibilities assigned; deliverables and timeline defined |
| <Date> | <Meeting / Activity> | <Attendees> | <Notes> |
| <Date> | <Meeting / Activity> | <Attendees> | <Notes> |

*Additional meeting minutes and group activity logs to be appended as the project progresses.*